Rockchip Linux Upgrade Solution Introduction

ID: RK-KF-YF-348

Release version: V1.3.0

Release Date: 2026-02-26

Security Level: □Top-Secret □Secret □Internal ■Public

DISCLAIMER

THIS DOCUMENT IS PROVIDED “AS IS”. ROCKCHIP ELECTRONICS CO., LTD.(“ROCKCHIP”)DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS,MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY. THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

Trademark Statement

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip’s registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

All rights reserved. ©2026. Rockchip Electronics Co., Ltd.

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, A District, No.89, software Boulevard Fuzhou, Fujian,PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

Overview

This document is intended to help engineers familiar with Rockchip Linux platform upgrade solution and do secondary development faster.

Intended Audience

This document (this guide) is mainly intended for:

Technical support engineers

Software development engineers

Revision History

Date	Version	Author	Revision Description
2019-06-05	V1.0.0	HKH/MLC	Initial version
2020-03-31	V1.0.1	ZLX	Update the format of the document
2020-12-14	V1.0.2	ZLX	Update the company name and document format
2021-02-18	V1.1.0	CWW	1. Update SD card boot to upgrade firmware 2. Update AB system upgrade
2023-03-06	V1.2.0	ZSQ	Add differential OTA function
2026-02-26	V1.3.0	ZJH	Added MTD/UBI differential upgrade support

Contents

Rockchip Linux Upgrade Solution Introduction

- 1. Overview and Codes
  - 1.1 Overview
  - 1.2 Codes
- 2. Recovery Mode
  - 2.1 Overview
  - 2.2 Configure and Compile
  - 2.3 OTA Upgrade
  - 2.4 Log Checking
- 3. Differential upgrade
  - 3.1 Dependencies and Building
  - 3.2 Partition size requirements
  - 3.3 Build differential package
  - 3.4 MTD/UBI Differential Upgrade
- 4. Linux A/B Mode
  - 4.1 Overview
  - 4.2 Boot Process
    - 4.2.1 Data Format and Storage
    - 4.2.2 Two boot modes
    - 4.2.3 Boot Process Flow Chart
  - 4.3 Compilation Configuration
    - 4.3.1 uboot
    - 4.3.2 Buildroot
    - 4.3.3 Partition Table
    - 4.3.4 Firmware Output
  - 4.4 OTA Upgrade
  - 4.5 Partition Boot Settings
    - 4.5.1 Bootable Settings
    - 4.5.2 Upgrade Partition Settings
- 5. Create a SD card Upgrade Disk
  - 5.1 Instructions for making SD card boot disk in Recovery system mode
  - 5.2 Instructions for Making SD Card Boot Disk in AB System Mode
    - 5.2.1 Prerequisites
    - 5.2.2 Creation Steps
- 6. Restore the Factory Settings
- 7. Details of the Upgrade Program
  - 7.1 Parameters
  - 7.2 Upgrade Customize Partition
- 8. Appendix
  - 8.1 Firmware Package Tools
    - 8.1.1 Windows Package Tool
    - 8.1.2 Linux Package Tool
  - 8.2 Misc Partition

1. Overview and Codes

1.1 Overview

Rockchip Linux platform supports two boot-up modes: recovery mode and Linux A/B mode:

1. Recovery mode: there is a separate partition (recovery.img) on the device for upgrade.
2. Linux A/B mode: there are two firmware on the device that can be switched.

Users can choose one according to their requirements for both the modes with their advantages and disadvantages.

1.2 Codes

Rockchip Linux platform supports two upgrade modes.

Upgrade Mode	path of codes	Recovery mode	Linux A/B mode
updateEngine	external/recovery/update_engine external/recovery	Support	Support
rkupdate	external/rkupdate	Support	Nonsupport

2. Recovery Mode

2.1 Overview

There will be a recovery partition on the device, which is consists of kernel+resource+ramdisk and mainly used for upgrade operations on the machine in recovery mode. U-boot will determine whether the system to be booted is a normal system or a recovery system based on the fields stored in the misc partition (see the misc partition section for details). Due to system independence, the recovery mode can guarantee the integrity of upgrade even when upgrade process is interrupted. If the power is abnormally powered down, the upgrade can still continue.

Advantages:

1. It can guarantee the integrity of upgrade.

Disadvantages:

1. There is one more partition which is only used for upgrade in the system.

2. Reboot the system into recovery mode during upgrade process and should not upgrade directly in the normal system.

Partition Instruction:

Partition name	Image name	Description
loader	MiniLoaderAll.bin	First level loader
u-boot	uboot.img	Second level loader
trust	trust.img	Secure environment, like OP-TEE, ATF(Some platforms will merge trust and uboot)
misc	misc.img	Boot parameter partition
recovery	recovery.img	Root file system consisting of kernel+dtb+ramdisk
boot	boot.img	kernel+dtb
rootfs	rootfs.img	Root file system, read only
oem	oem.img	used for manufacturer, read and write
userdata	userdata.img	used for users, read and write

2.2 Configure and Compile

Buildroot: the recovery configuration files are selected as follows: (make menuconfig):

```
BR2_PACKAGE_RECOVERY=y  #turn on upgrade related functions
BR2_PACKAGE_RECOVERY_USE_UPDATEENGINE=y #use the new upgrade program, if not configured,
the original upgrade process will be used by default.
BR2_PACKAGE_RECOVERY_RECOVERYBIN=y  #turn on recovery bin file
BR2_PACKAGE_RECOVERY_UPDATEENGINEBIN=y #compile the new upgrade program
BR2_PACKAGE_RECOVERY_NO_UI=y # disable UI
```

Buildroot: the rootfs configuration files are selected as follows (make menuconfig):

```
BR2_PACKAGE_RECOVERY=y #turn on upgrade related functions
BR2_PACKAGE_RECOVERY_USE_UPDATEENGINE=y #use the new upgrade program
BR2_PACKAGE_RECOVERY_UPDATEENGINEBIN=y  #compile the new upgrade program
BR2_PACKAGE_RECOVERY_NO_UI=y # disable UI
```

With and Without Display

Currently only RK3308 uses the recovery without display, if the recovery of other chipset do not need display, just configure the following item in the file "buildroot/package/rockchip/recovery/recovery":

```
TARGET_MAKE_ENV += RecoveryNoUi=true
```

The SDK will enable the above configuration by default, so users do not need to configure it again. The source code is in "external/recovery/" directory, if you need to do some modifications, compile as follows:

```
# Step1: run the following command:
source envsetup.sh
# Step2: choose the rootfs configuration of a platform,and then run the following command:
make recovery-dirclean
source envsetup.sh
# Step3: choose the recovery configuration of a platform, and then run the following
command:
make recovery-dirclean
./build.sh
# Step4: flashing the firmware again.
```

If the SDK version is relatively new, you can try to compile as follows:

```
./build.sh external/recovery
./build.sh
# flashing the firmware again.
```

2.3 OTA Upgrade

The upgrade supports network upgrade and local upgrade, and you can specify the partition to be upgraded. Run the following command on the normal system.

Network upgrade:

```
# updateEngine --misc=update --image_url=firmware address --partition=0x3FFC00 --
version_url=version file address --savepath=/userdata/update.img --reboot
updateEngine --image_url=http://172.16.21.110:8080/recovery/update.img \
    --misc=update --savepath=/userdata/update.img --reboot &
```

Local upgrade:

```
updateEngine --image_url=/userdata/update.img --misc=update \
    --savepath=/userdata/update.img --reboot &
```

Process introduction:

1. Firmware versions comparison(--version_url)
2. Download the firmware(--image_url) and save it locally(--savepath)
3. Upgrade recovery partition
4. Reboot(--reboot)
5. Enter the recovery mode, upgrade the partition(--partition)
6. Successfully upgrade and reboot into the normal system.

Optional parameters:

1. --version_url: remote address or local address. If this parameter is not set, versions comparison will not do .
2. --savepath: firmware saves path, it is "/tmp/update.img" by default. It is recommended to set to "/userdata/update.img".
3. --partition: set the partition to be upgraded. It is recommended to set it to 0x3FFC00. **It is not supported to upgrade the parameter and loader partitions**. See the Chapter of Parameters Introduction for details.
4. --reboot: after upgrading the recovery partition, reboot into the recovery mode.

2.4 Log Checking

1. Check log by serial port

In the "buildroot/output/rockchip_***/target" directory:

```
touch .rkdebug
```

Create this hidden file to print the upgraded log in recovery mode by serial port.

2. Checking through the "userdata/recovery/Log" file:

After upgrading, check the log file under "userdata/recovery" directory in the device.

```
cat userdata/recovery/Log
```

3. Differential upgrade

The `tools/linux/Linux_Diff_Firmware/` under Linux SDK can be used to make differential package, which has features as below:

- support uboot/trust/kernel partitions
- support squashfs rootfs partition (including block devices and UBI volumes)
- support MTD-based NAND flash systems (both raw MTD partitions and UBI SquashFS partitions)

and features are not ready yet like:

- the read/write partitions, e.g. EXT2/3/4 filesystem
- the encrypted partitions
- A/B mode OTA

3.1 Dependencies and Building

The tools that the PC side relies on include: bsdiff, md5sum, ubireader_extract_images. They can be installed using the following commands:

```
sudo apt install bsdiff
pip install ubi_reader
```

The `ubi_reader` package is used to extract squashfs data from UBI images when building differential packages. If rootfs.img is in UBI image format, this tool must be installed, otherwise the differential package build will fail.

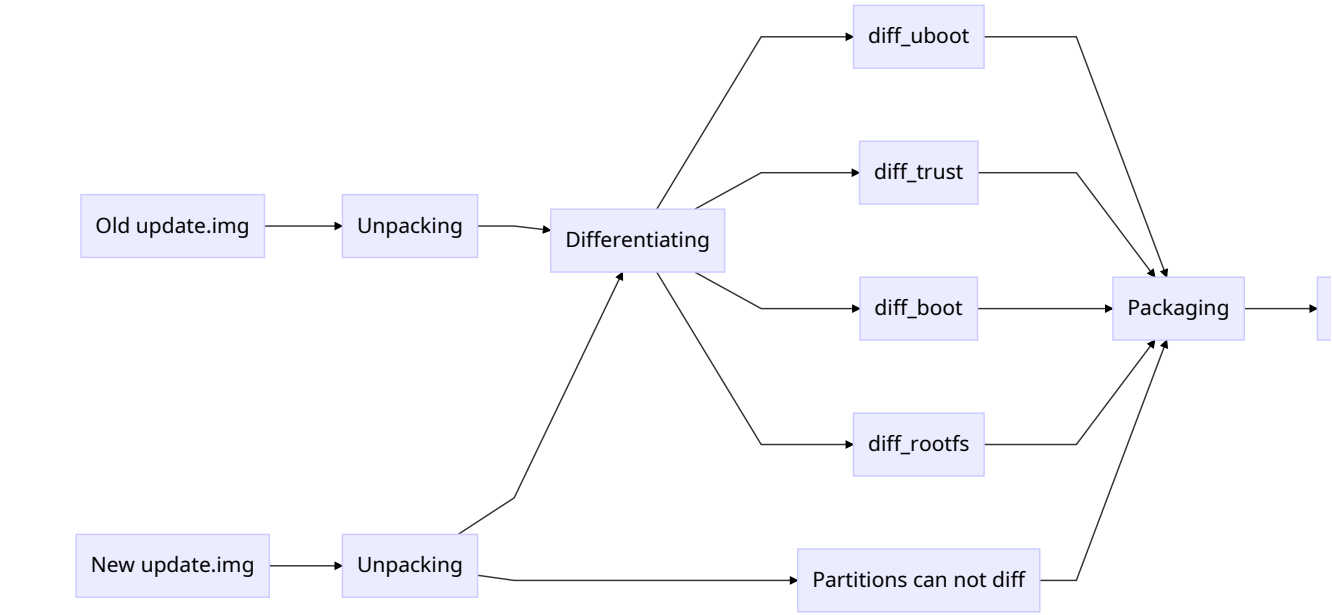
3.2 Partition size requirements

Requirements for ddr and partition size at the board end,

- No requirement for DDR size, regardless of the size of the differential package
- The size of free partitions should not be less than the sum of update_diff.img and rootfs.img file size
    - For example, saying update_diff.img is saved in the /userdata/ directory, then
    - At the very least, it is necessary to be able to save the new rootfs.img image in /userdata

3.3 Build differential package

Create a differential package by unpacking, differentiating the new and old OTA images, and packaging.



NOTE

- It's better the new update.img only contains the partition to be upgraded
- For old firmware, it is recommended to use update.img that contains all partitions, and at least contain partitions to be upgraded
- The old firmware must be the same as the firmware on the target machine

Usage:

```
cd tools/linux/Linux_Diff_Firmware/
./mk-diff-ota.sh update_old.img update_new.img update_diff.img
```

It will generate update_diff.img which can replace the new update.img on target board for upgrading.

3.4 MTD/UBI Differential Upgrade

For NAND flash systems using the MTD driver, differential upgrade supports the following two partition types:

- **Raw MTD partitions** (e.g. boot): During upgrade, old data is read from the MTD device, the bsdiff patch is applied, then written back via flash_erase + nandwrite.
- **UBI SquashFS partitions** (e.g. rootfs): During upgrade, the UBI device is attached first, old squashfs data is read from the UBI volume, the bsdiff patch is applied to generate the new squashfs, then written to the UBI volume via ubiupdatevol.

When building differential packages, if rootfs.img is in UBI image format, the script automatically uses `ubireader_extract_images` to extract squashfs data from the UBI image, then performs the diff against the new squashfs. This ensures the diff baseline data is consistent with the data stored in the UBI volume on the device.

NOTE

- shall use `updateEngine` on target board to do upgrading, the `rkupdate` does not support differential package yet.

4. Linux A/B Mode

4.1 Overview

You have to prepare two separate system firmwares which are respectively stored in the flash in Linux A/B mode . The system can be booted from one of the slots. If the current slot fails to boot, it can be started from another slot, and the system can be directly upgraded in the normal state, without entering the system upgrade mode, and just restart the system to enter the upgraded system.

The following **advantages** are included in Linux A/B mode due to its two boot slots:

1. The machine can be upgraded directly on the current system without rebooting into upgrade mode.
2. Prevent the machine from crashing for failure upgrade. If the upgrade fails, the machine can return to the current version.
3. If the current system is destroyed due to misoperation, the system will automatically switch to another slot.

Disadvantages:

1. There are two slots in Linux A/B mode, so it will increase the system firmware usage on the flash.

Partition:

Since there are already multiple backups of miniloader, trust, and uboot on the device, these partitions do not support the dual partitions at present, only the boot and system are dual-partitioned. The partition table is as follows:

Partition	Image	Description
loader	Miniloader.bin	First level loader, there are 4 backups in the device
uboot_a	uboot.img	Second level loader, there are 2 backups in the device, the backups number can be changed by modifying the "u-boot/make.sh"
uboot_b	uboot.img	backup of uboot_a
trust	trust.img	Related to secure, there are 2 backups in the device, the backups number can be changed by modifying the "u-boot/make.sh" (Some platforms will merge trust and uboot)
misc	misc.img	Boot parameter partition
boot_a	boot.img	kernel+dtb, boot system_a
boot_b	boot.img	kernel+dtb, boot system_b
system_a	rootfs.img	Root file system
system_b	rootfs.img	Root file system
oem	oem.img	Manufacturer prefabricated, read and write, No backup
userdata	userdata.img	For data, readable and writable, No backup

4.2 Boot Process

4.2.1 Data Format and Storage

The storage position is 2K offset from the misc partition, and AvbABSlotData and AvbABData data structures are as follows

AvbABSlotData: store slot_a and slot_b

Date name	Usage
unsigned char priority	Partition priority, 0~15, 0 is not automatic, 15 is the highest priority
unsigned char tries_remaining	The number of attempts to start, the highest is 7, can be modified
unsigned char successful_boot	0: unbootable, 1: bootable
unsigned char is_update:1	0: upgrade failed, 1: upgrade successfully , and the last 7 bits are reserved data.

AvbABData: slot_a and slot_b boot information

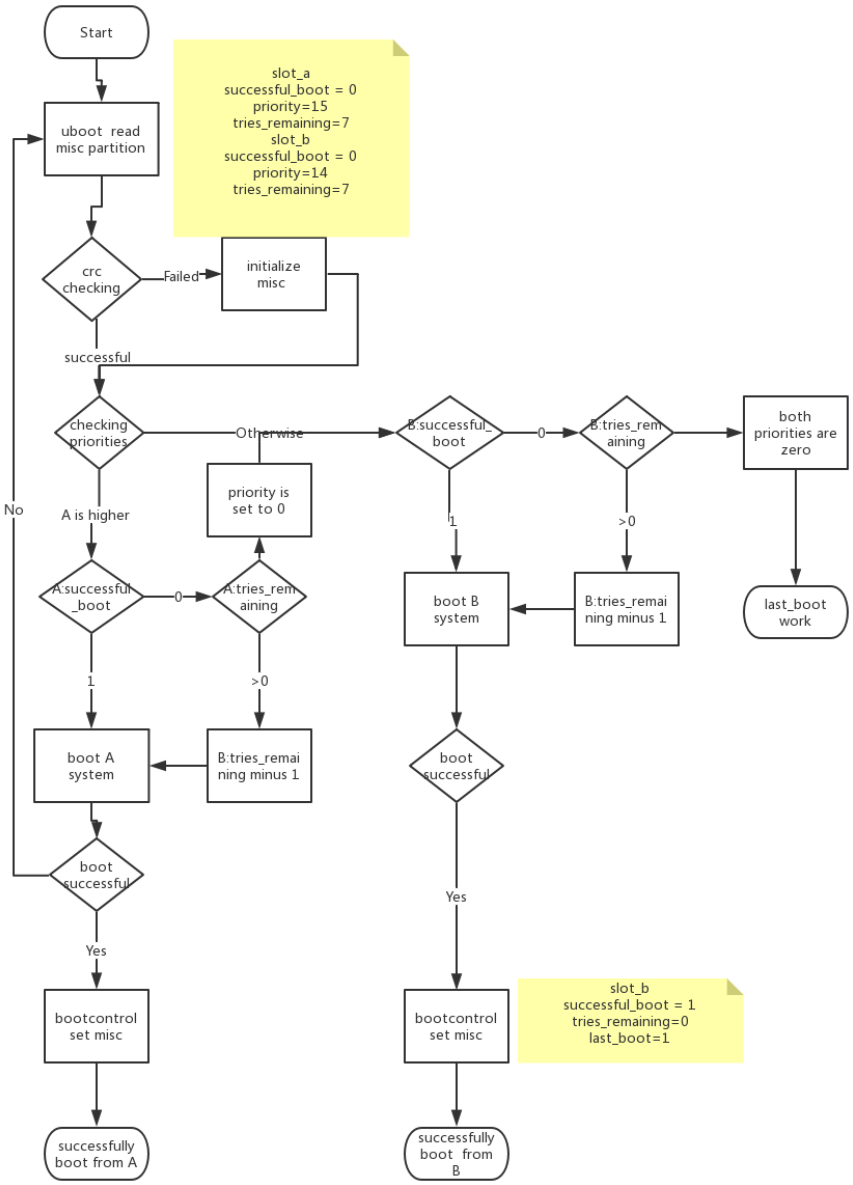
Date name	Usage
unsigned char magic[AVB_AB_MAGIC_LEN]	structure header information: \0AB0
unsigned char version_major	version information
unsigned char version_minor	version information
unsigned char reserved1[2]	reserved data
AvbABSlotData slots[2]	Partition boot information
unsigned char last_boot	The partition that was successfully boot last time: 0->slot_a, 1->slot_b
unsigned char reserved2[11]	reserved data
unsigned char crc32	Crc data verification

4.2.2 Two boot modes

According to the configuration method of the upper bootcontrol program, there are two boot modes: successful_boot and reset retry. The difference of the two modes is as follows:

Mode	Advantage	Disadvantage	parameters need to set when successfully boot(A boot)	parameters need to set when upgrade(A boot, B upgrade)
Successful boot	As long as the system is booted normally, it will not go back to the old version of firmware.	After the device is working for a long time, if some memory are stored abnormally, the system will always restart.	tries_remaining=0 successful_boot=1 last_boot=0	A:priority=14 B:priority=15
Reset retry	Always maintain the retry mechanism can fix storage issues	1. The machine will go back to the old version which may not be controlled. 2.If the retry try many times for users misoperation, it will be misidentified that the current partition is bootable.	tries_remaining=7 last_boot=0	A:priority=14 B:priority=15

4.2.3 Boot Process Flow Chart



4.3 Compilation Configuration

4.3.1 uboot

Add the following configuration to defconfig, such as u-boot/configs/rk-ab.config

```
CONFIG_SPL_AB=y
CONFIG_AVB_LIBAVB=y
CONFIG_AVB_LIBAVB_AB=y
CONFIG_AVB_LIBAVB_ATX=y
CONFIG_AVB_LIBAVB_USER=y
CONFIG_RK_AVB_LIBAVB_USER=y
CONFIG_ANDROID_AB=y
```

4.3.2 Buildroot

```
BR2_PACKAGE_RECOVERY=y  #enable recovery function
BR2_PACKAGE_RECOVERY_BOOTCONTROL=y      #enable the boot control script
BR2_PACKAGE_RECOVERY_RETRY=y
#The boot mode is retry mode. it is successful_boot mode by default when not configured.
BR2_PACKAGE_RECOVERY_USE_UPDATEENGINE=y  #Use the new upgrade program
BR2_PACKAGE_RECOVERY_UPDATEENGINEBIN=y
BR2_PACKAGE_RECOVERY_NO_UI=y # disable UI
#compile the new upgrade program
```

Note: after finishing the configuration, you must recompile as follows:

```
1. cd buildroot
2. source envsetup.sh
3. Select the rootfs configuration for a specific platform.
4. make recovery-recreate
```

4.3.3 Partition Table

The following is an example of creating and editing parameter partition information:

```
CMDLINE:
mtdparts=:0x00004000@0x00004000(uboot_a),0x00004000@0x00008000(uboot_b),0x00002000@0x0000c000
(misc),0x00020000@0x0000e000(boot_a),0x00040000@
0x0002e000(boot_b),0x00010000@0x0006e000(backup),0x00e00000@0x0007e000(system_a),0x00e00000@0
x00e7e000(system_b),0x00040000@0x01c7e000(oem),-@0x01cbe000(userdata:     grow)
```

The rootfs filesystem needs to be named system_a/system_b.

The corresponding defconfig parameter for the partition table should be set as follows:

```
RK_AB_UPDATE=y
RK_PARAMETER="parameter-ab.txt"
```

4.3.4 Firmware Output

After selecting the corresponding board configuration, run:

```
./build.sh
```

The following firmware images will be generated:

```
rockdev/
├── boot.img
├── MiniLoaderAll.bin
├── misc.img
├── oem.img
├── parameter.txt
├── rootfs.img
├── uboot.img
├── update-ab.img
├── update.img
└── userdata.img
```

> **Note:** If `RK_SDCARD_update` is configured in defconfig (pointing to a non-AB partition table with a recovery partition in the chip directory) and `recovery.img` has been built, the SDK will automatically generate `update-sd.img` during the packaging phase for creating SD card boot disks. See Instructions for Making SD Card Boot Disk in AB System Mode for details.

Firmware Upgrade

```
./build.sh ota-updateimg
```

In the `rockdev` and `IMAGE` directories, an `update_ota.img` file will be generated for OTA upgrades. This `IMAGE` package contains `boot.img` and `rootfs.img`. You can modify the `./build.sh edit-ota-package-file` file of the firmware to be packaged according to your actual needs. As shown below:

```
# NAME          Relative path
#
#HWDEF         HWDEF
package-file   package-file
bootloader     Image/MiniLoaderAll.bin
parameter      Image/parameter.txt
uboot_a        Image/uboot.img
boot_a         Image/boot.img
system_a       Image/rootfs.img
oem            Image/oem.img
```

Firmware flashing

In the `rockdev` and `IMAGE` directories, `update-ab.img` will be generated. This firmware is used for flashing. Modify the firmware to be packaged according to your needs: `./build.sh edit-package-file`. As shown below:

```
# NAME          Relative path
#
#HWDEF         HWDEF
package-file   package-file
bootloader     Image/MiniLoaderAll.bin
parameter      Image/parameter.txt
misc           Image/misc.img
uboot_a        Image/uboot.img
uboot_b        Image/uboot.img
```

```
boot_a          Image/boot.img
boot_b          Image/boot.img
system_a        Image/rootfs.img
system_b        Image/rootfs.img
oem             Image/oem.img
```

4.4 OTA Upgrade

Upgrade online:

```
# updateEngine --update --image_url=firmware address --partition=0x3FFC00 --
version_url=version file address --savepath=save the firmware address --reboot
updateEngine --image_url=http://172.16.21.110:8080/linuxab/update_ota.img --update --reboot
```

Upgrade locally:

```
# updateEngine --update --image_url=firmware address(update-ab.img or update_ota.img) --
partition=0x3FFC00 --version_url=version file address --savepath=save the firmware address --
reboot
updateEngine --image_url=/userdata/update_ota.img --update --reboot
```

Process introduction:

1. Firmware versions comparison
2. Download the firmware (--image_url) and save it locally (--savepath)
3. Upgrade the specified partition(--partition)
4. Set the upgrade partition to the partition to be upgraded
5. Restart
6. Try to boot the upgraded partition

Optional parameters:

1. --partition: set the partition to be upgraded. In Linux A/B mode, it is recommended to upgrade only uboot_a/uboot_b boot_a/boot_b and system_a/system_b, that is setting it to 0xFC00. **It is not supported to upgrade the parameter and loader partitions**. See parameter instruction for details.
2. --version: if this parameter is not set, there will be no versions comparison.
3. --savepath: firmware save path. It is "/tmp/update.img" by default. and it is recommended to use the default value.
4. --reboot: reboot after upgrade

4.5 Partition Boot Settings

4.5.1 Bootable Settings

Set the current partition to be bootable by the misc and then execute after the system successfully boots, mark the system to start successfully. Please refer to the following script.

```
$external/recovery/update_engine$ cat S99_bootcontrol
case "$1" in
  start)
        /usr/bin/updateEngine --misc=now
        ;;
  stop)
        printf "stop finished\n"
        ;;
  *)
        echo "Usage: $0 {start|stop}"
        exit 1
        ;;
esac
exit 0
```

4.5.2 Upgrade Partition Settings

```
updateEngine --misc=other --reboot
```

Process introduction:

1. Write a command to the position 4K offset from the misc in order to boot another partition
2. Restart

Optional parameter:

1. --reboot, the machine will not restart immediately and will not take effect until the next reboot by default.

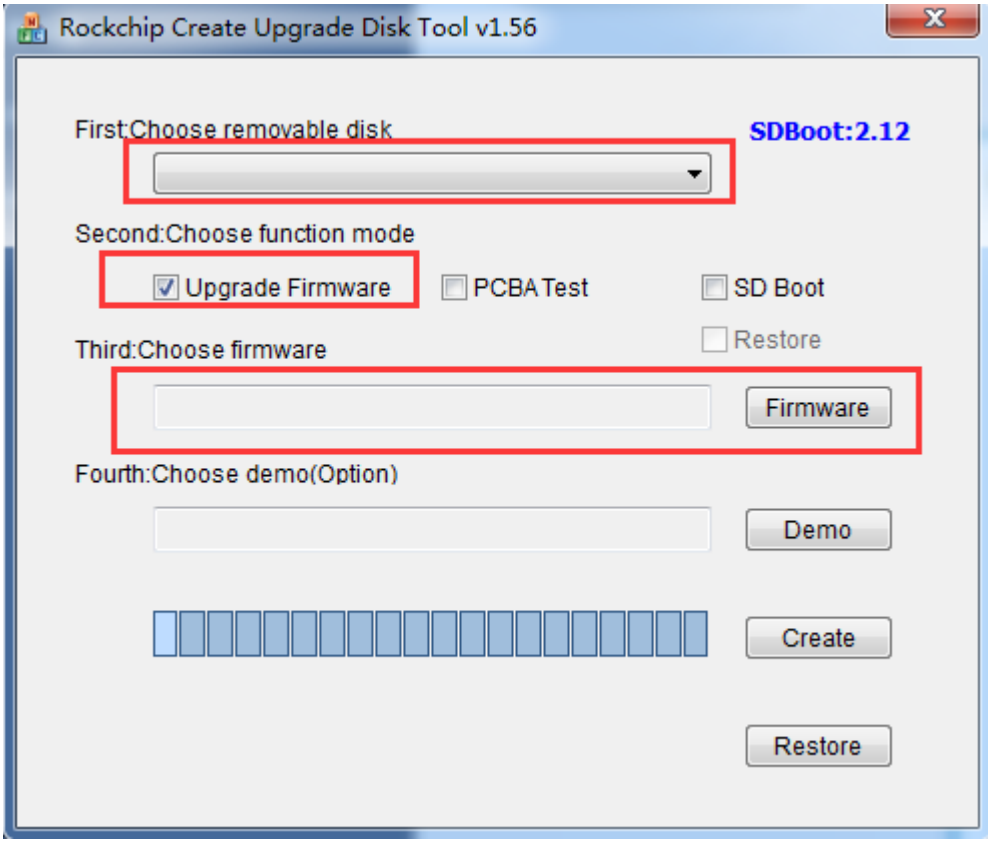
Note: the updateEngine program is automatically set after OTA upgrade is completed, no need to repeat the settings.

5. Create a SD card Upgrade Disk

Creating an upgrade disk refers to inserting the SD card created by the SDDiskTool card-making tool to the machine to upgrade, this section will introduce how to create a upgrade disk and some upgrade issues in details.

Create an Upgrade Disk

As shown in the figure below, use the tool in the project directory "tools\windows\SDDiskTool" to create a upgrade disk.



Choose the packaged update.img file by the "Firmware" button.

After that, click the "Create" button. If the creation is successful, a prompt will appear.

At this point, there will be two files in the root directory of the SD card, and the upgraded firmware update.img will be named sdupdate.img.

After all the preparations are completed, insert the SD card into the device and power on.

File System Format Notes:

By default, SDDiskTool formats the SD card with the FAT32 file system, which has the following limitations:

- Maximum single file size: 4GB
- Maximum SD card capacity: 32GB

If the upgrade package exceeds 4GB or the SD card capacity exceeds 32GB, creating the boot disk with FAT32 will fail. In this case, modify the `config.ini` configuration file in the SDDiskTool directory to use exFAT or another file system that supports large files:

```
USER_DISK_FS=EXFAT
```

> **Note:** After changing the file system format, ensure that the device kernel has enabled support for the corresponding file system (e.g., `CONFIG_EXFAT_FS=y`), otherwise the device will not be able to mount the SD card.

For the SD boot card format details, refer to *Rockchip_Developer_Guide_UBoot_Nextdev_EN.pdf*.

5.1 Instructions for making SD card boot disk in Recovery system mode

To make SD card boot disk in Recovery system mode, just use SDDiskTool to make update.img directly into sdupdate.img.

5.2 Instructions for Making SD Card Boot Disk in AB System Mode

SD card boot disk upgrade relies on Recovery mode for bootstrapping. Since the AB system partition table does not include a recovery partition, the device cannot directly boot into Recovery mode from an SD card to perform the upgrade. To address this, the SDK automatically generates `update-sd.img` based on the non-AB partition table (which includes the recovery partition) during the packaging phase, specifically for creating SD card boot disks.

5.2.1 Prerequisites

1. `RK_SDCARD_UPDATE=y` is configured in defconfig.
2. `recovery.img` has been built (run `./build.sh recovery` to build it).

Configuration example:

```
RK_PARAMETER="parameter-ab.txt"
RK_AB_UPDATE=y
RK_SDCARD_UPDATE=y
```

When the above conditions are met, executing `./build.sh updateimg` will automatically generate the following firmware images:

- **update-ab.img**: Packed with the AB partition table, contains the complete dual-slot system images for device flashing.
- **update-sd.img**: Packed with the non-AB partition table (including the recovery partition), used exclusively for creating SD card boot disks.

> **Note:** If `RK_SDCARD_UPDATE` is not configured, the generation of `update-sd.img` will be skipped.

5.2.2 Creation Steps

1. Use SDDiskTool to load `update-sd.img` and create the SD card boot disk.
2. Copy `update-ab.img` to the SD card root directory and rename it to `sdupdate.img`.
3. Insert the SD card into the target device and power on. The device will automatically boot into Recovery mode and perform the upgrade.

After creation, the SD card root directory structure is as follows:

```
rksdfw.tag
sd_boot_config.config
sdupdate.img            # renamed from update-ab.img
```

> **Note:** The recovery partition in `update-sd.img` is solely used to bootstrap the SD card upgrade process. The actual firmware written to the target device (`sdupdate.img`) uses the standard AB partition table, and the target device will not contain a recovery partition after the upgrade.

6. Restore the Factory Settings

The configuration files that can be read and written were stored in the userdata partition. The factory firmware will set some configuration parameters by default. After using a period of time, the configuration file will be generated or modified. Sometimes users need to clear the data, at this time, they need to restore the factory configuration.

SDK implementation:

Function keys "RECOVERY + VOLUMEUP" are used to trigger to restore the factory configuration, please refer to the code in these directory:

buildroot/board/rockchip/rk3308/fs-overlay/etc/input-event-daemon.conf

board/rockchip/rk3308/fs-overlay/usr/sbin/factory_reset_cfg

```
updateEngine --misc=wipe_userdata --reboot
```

Process introduction:

1. Write format command to the position that 4k offset from the misc partition
2. Reboot(--reboot)
3. The S21mountall.sh is used to dentify formatting commands in the misc
4. format the userdata

Optional parameter:

1. --reboot: if this parameter was not set, the factory settings will be restored after the next reboot of the machine.

7. Details of the Upgrade Program

7.1 Parameters

The updateEngine mainly includes upgrading partition and writing the Misc configuration function, command parameters are as follows:

```
*** update_engine: Version V1.1.0 ***.
--misc=now              Linux A/B mode: Setting the current partition to bootable.
--misc=other            Linux A/B mode: Setting another partition to bootable.
--misc=update           Recovery mode: Setting the partition to be upgraded.
```

```
--misc=wipe_userdata   Format data partition.
--update               Upgrade mode.
--partition=0x3FFC00   Set the partition to be upgraded.(NOTICE: OTA not support upgrade
loader and parameter)
                       0x3FFC00: 0011 1111 1111 1100 0000 0000.
                                 uboot trust boot recovery rootfs oem
                                 uboot_a uboot_b boot_a boot_b system_a system_b.
                       000000000000000000000000: reserved
                       100000000000000000000000: Upgrade loader
                       010000000000000000000000: Upgrade parameter
                       001000000000000000000000: Upgrade uboot
                       000100000000000000000000: Upgrade trust
                       000010000000000000000000: Upgrade boot
                       000001000000000000000000: Upgrade recovery
                       000000100000000000000000: Upgrade rootfs
                       000000010000000000000000: Upgrade oem
                       000000001000000000000000: Upgrade uboot_a
                       000000000100000000000000: Upgrade uboot_b
                       000000000010000000000000: Upgrade boot_a
                       000000000001000000000000: Upgrade boot_b
                       000000000000100000000000: Upgrade system_a
                       000000000000010000000000: Upgrade system_b
                       000000000000001000000000: Upgrade misc
                       000000000000000100000000: Upgrade userdata
--reboot               Restart the machine at the end of the program.
--version_url=url      The path to the file of version.
--image_url=url        Path to upgrade firmware.
--savepath=url         save the update.img to url.
```

--misc

now: used to set the current partition as a bootable partition in Linux A/B mode

Note: The "external/recovery/update_engine/S99bootcontrol" script will run this command in the end of booting, setting the current partition as a bootable partition, which needs to be enabled.

```
BR2_PACKAGE_RECOVERY_BOOTCONTROL=y
```

other: used to set another partition as the upgraded partition, and try to boot from another partition after reboot in Linux A/B mode.

Note: if the updateEngine is used to upgrade, it will be set automatically after the upgrade is completed, no need to repeat the settings.

update: used to upgrade the recovery partition in the normal system, and upgrade the remaining partitions in the recovery system in Recovery mode.

display: used to debug, display the data structure of the misc partition.

--update

sdboot: do the sdboot upgrade process, that is, operate flash directly without partition concept.

Without parameters: mainly used in Linux A/B mode, directly upgrade in the current mode.

--partition=0x0000

Set the partition to be upgraded, the default value is 0x3FFC00, upgrade uboot, trust, boot, recovery, rootfs, oem, uboot_a, uboot_b, boot_a, boot_b, system_a, system_b partitions. The higher 16bits are already used, and the lower 8 bits are reserved bits which can be extended.

1: upgrade, 0: not upgraded

bits	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7~0
partition	loader	parameter	uboot	trust	boot	recovery	rootfs	oem	uboot_a	uboot_b	boot_a	boot_b	system_a	system_b	misc	userdata	reserved

--reboot

After the updateEngine runs successfully, the machine restarts.

--vsersion_url

If it was set, it will be compared to the RK_VERSION= version value in the "/etc/version" file before upgrade.

Local path: read the version number from the firmware.

Remote path: download the version file from the remote, and the remote version file format must be consistent with the "/etc/version".

--image_url

Set the path of the upgrade firmware, which can be remote or local path.

--savepath

Set the path where the firmware is saved. If it is not set and the upgraded firmware path is a remote address, the default value is /tmp/update.img.

7.2 Upgrade Customize Partition

```
typedef struct {
char name[32];          //firmware name
bool need_update;       //need to upgrade
bool is_ab;                     //whether it is A/B dual patition
long long size;         //firmware size
long long offset;       //Offset position in update.img
long long flash_offset; //Offset position in flash
char dest_path[100];    //Target path
update_func cmd;                //Upgrade function
}UPDATE_CMD, *PUPDATE_CMD;
```

To upgrade the customize partition "factory", add the following line below, and --partition needs to set the corresponding bit value to 1:

```
{"factory", false, false, 0, 0, 0, "", flash_normal},
```

external/recovery/update_engine/update.cpp

```
UPDATE_CMD update_cmd[] = {
           {"bootloader" , false , false , 0 , 0 , 0 , "" , flash_bootloader} ,
           {"parameter"  , false , false , 0 , 0 , 0 , "" , flash_parameter}  ,
           {"uboot"      , false , false , 0 , 0 , 0 , "" , flash_normal}     ,
           {"trust"      , false , false , 0 , 0 , 0 , "" , flash_normal}     ,
           {"boot"       , false , true  , 0 , 0 , 0 , "" , flash_normal}     ,
           {"recovery"   , false , false , 0 , 0 , 0 , "" , flash_normal}     ,
           {"rootfs"     , false , true  , 0 , 0 , 0 , "" , flash_normal}     ,
           {"oem"        , false , false , 0 , 0 , 0 , "" , flash_normal}     ,
           {"uboot_a"    , false , false , 0 , 0 , 0 , "" , flash_normal}     ,
           {"uboot_b"    , false , false , 0 , 0 , 0 , "" , flash_normal}     ,
           {"boot_a"     , false , false , 0 , 0 , 0 , "" , flash_normal}     ,
           {"boot_b"     , false , false , 0 , 0 , 0 , "" , flash_normal}     ,
           {"system_a"   , false , false , 0 , 0 , 0 , "" , flash_normal}     ,
           {"system_b"   , false , false , 0 , 0 , 0 , "" , flash_normal}     ,
           {"misc"       , false , false , 0 , 0 , 0 , "" , flash_normal}     ,
           {"userdata"   , false , false , 0 , 0 , 0 , "" , flash_normal}     ,
```

8. Appendix

8.1 Firmware Package Tools

8.1.1 Windows Package Tool

The Windows package tool is located in the tools\windows\AndroidTool\rockdev directory. Modify the package-file first to add the image to be upgraded into package. Note that the path here is relative path.

The mkupdate.bat batch program will link the "tools\windows\AndroidTool\rockdev\Image" to the rockdev directory under root directory. So please make sure the corresponding image exists under rockdev. Then execute the mkupdate.bat which will package corresponding image in rockdev under root directory into update.img and stored in rockdev under root directory.

8.1.2 Linux Package Tool

The Linux package tool is available in the "tools/linux/Linux_Pack_Firmware/rockdev" directory. Modify the package-file first to add the image to be upgraded into package. Note that the path here is relative path.

The "tools/linux/Linux_Pack_Firmware/rockdev/Image" will link to the rockdev under root directory. So please make sure the corresponding image under rockdev exists. Then execute the mkupdate.sh which will package the corresponding image in rockdev under root directory into update.img and stored in rockdev under root directory.

8.2 Misc Partition

The Misc partition is a partition without a file system. It is used to store some boot configuration parameters. The current structure is as follows. Please see the "external/recovery/bootloader.h" and "external/recovery/update_engine/rkbootloader.cpp" for details.

Offset address	Usage
2k	Linux A/B partition boot information
4k	Format command
16k	Communication between Recovery system and Normal system